LAPORAN PRAKTIKUM

POSTTEST 8

ALGORITMA PEMROGRAMAN DASAR



Disusun oleh:

Gian Ananta Prabasuara

2609106081

Kelas C1 ‘26

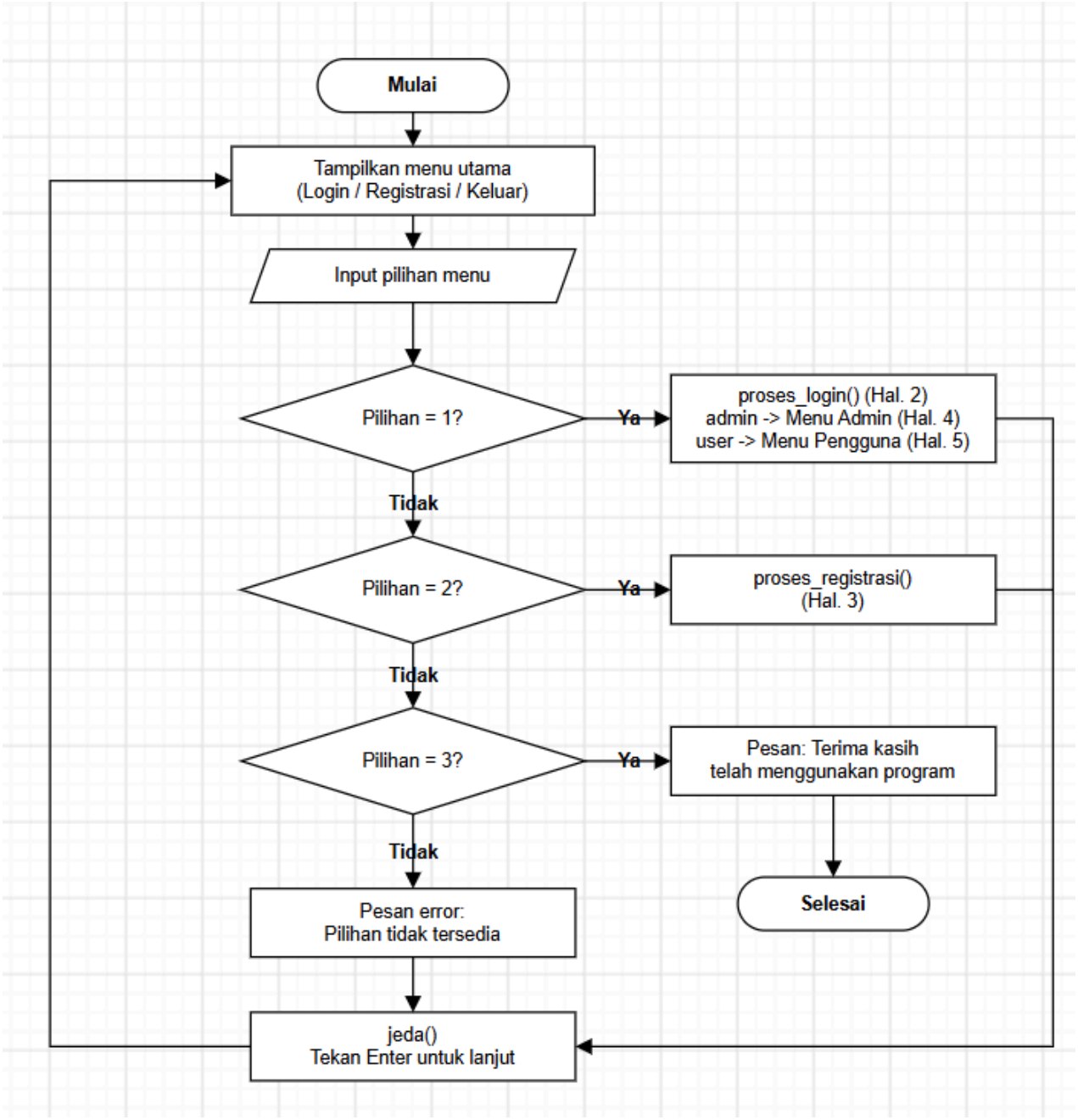
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

SAMARINDA

2026

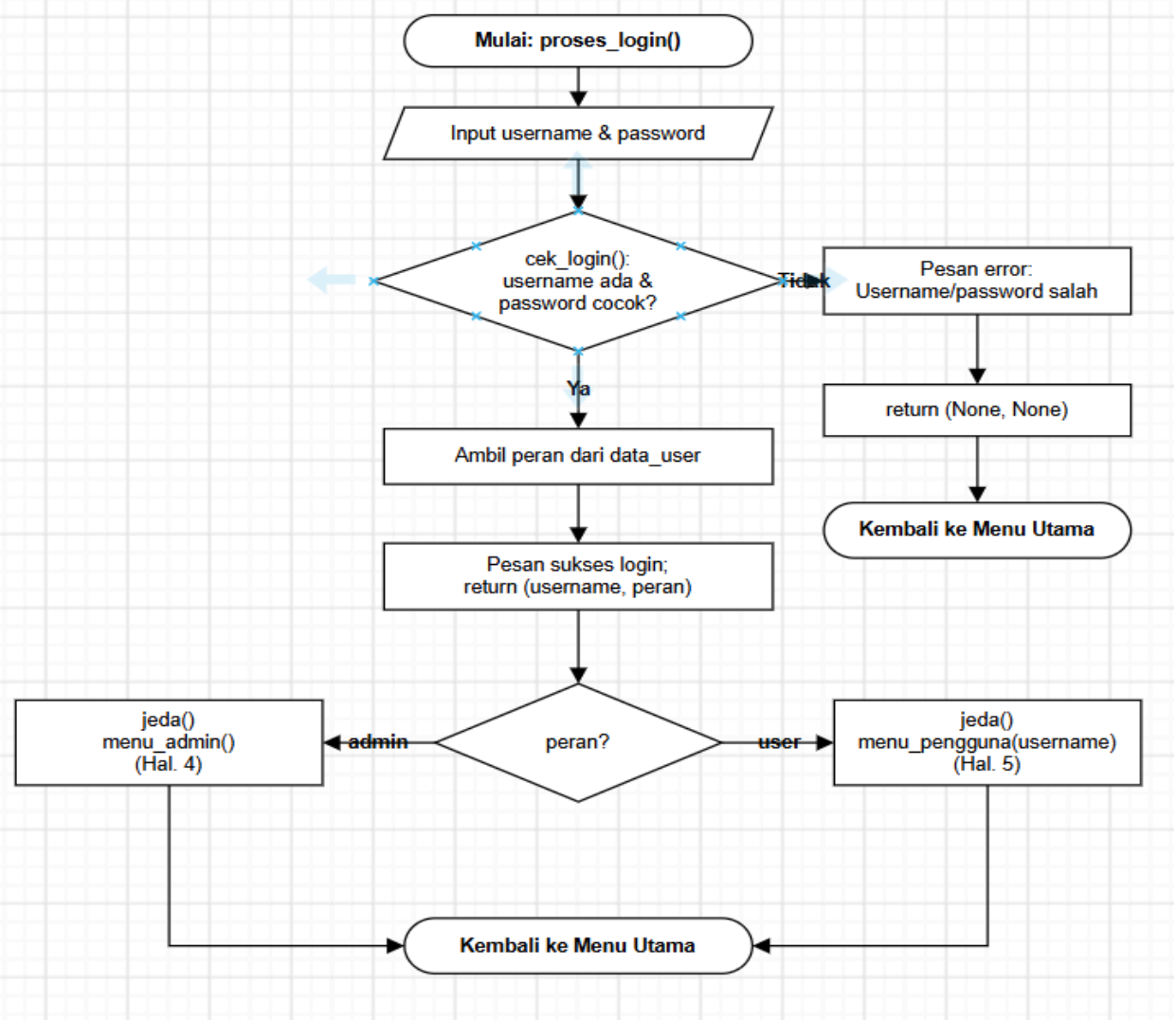
1. Flowchart



Gambar 1.1 Menu Utama

1. Menu Utama

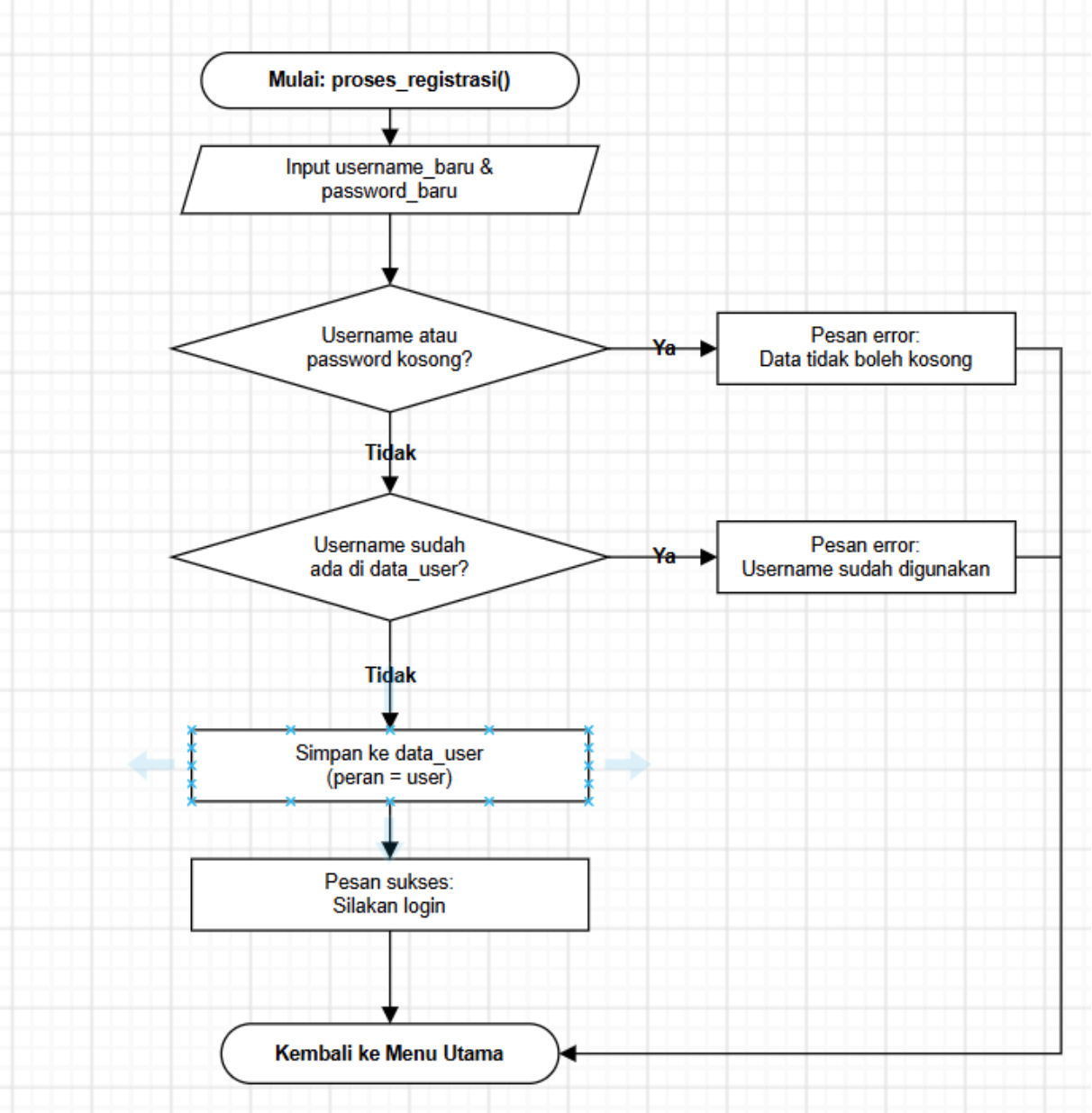
1. **Input:** Pengguna memasukkan pilihan menu (1, 2, 3).
2. **Proses:** Program melakukan inisialisasi data awal dan mengevaluasi pilihan menggunakan struktur kondisi (if-else).
3. **Output:** Mengarahkan ke menu Login (jika pilih 1), Registrasi (jika pilih 2), keluar dri program (jika pilih 3), atau menampilkan pesan "Pilihan tidak tersedia" jika inputnya salah.



Gambar 1.2 Login

2. Login

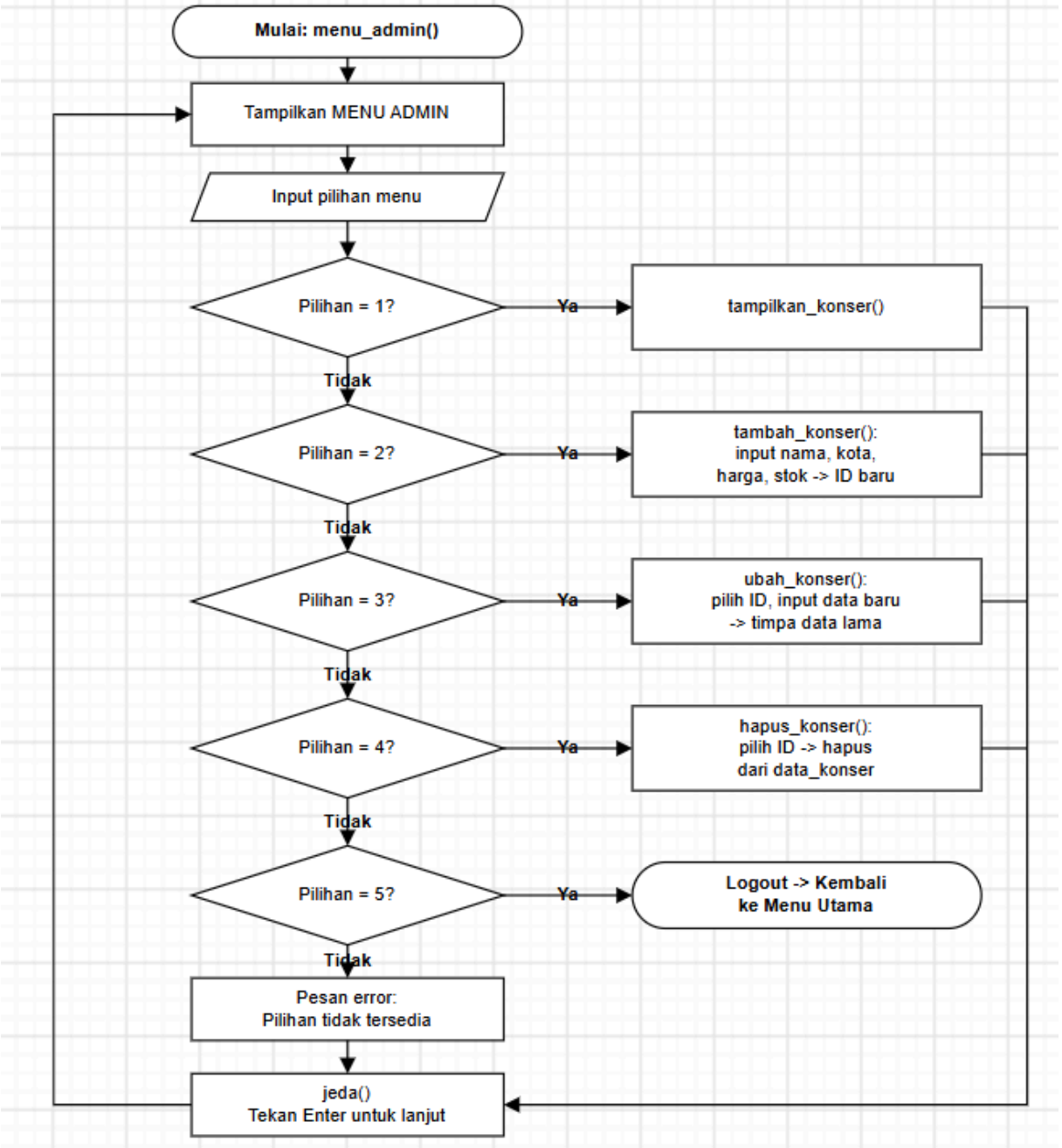
1. **Input:** Pengguna memasukkan username dan password.
2. **Proses:** Program memeriksa kecocokan data dengan data user dan menentukan status login dan peran
3. **Output:** Kalau gagal, mencetak pesan error dan kembali ke Menu Utama; jika berhasil, akan menampilkan pesan sukses dan mengarahkan ke Menu Admin atau Menu Pengguna berdasarkan perannya.



Gambar 1.3 Registrasi

3. Registrasi

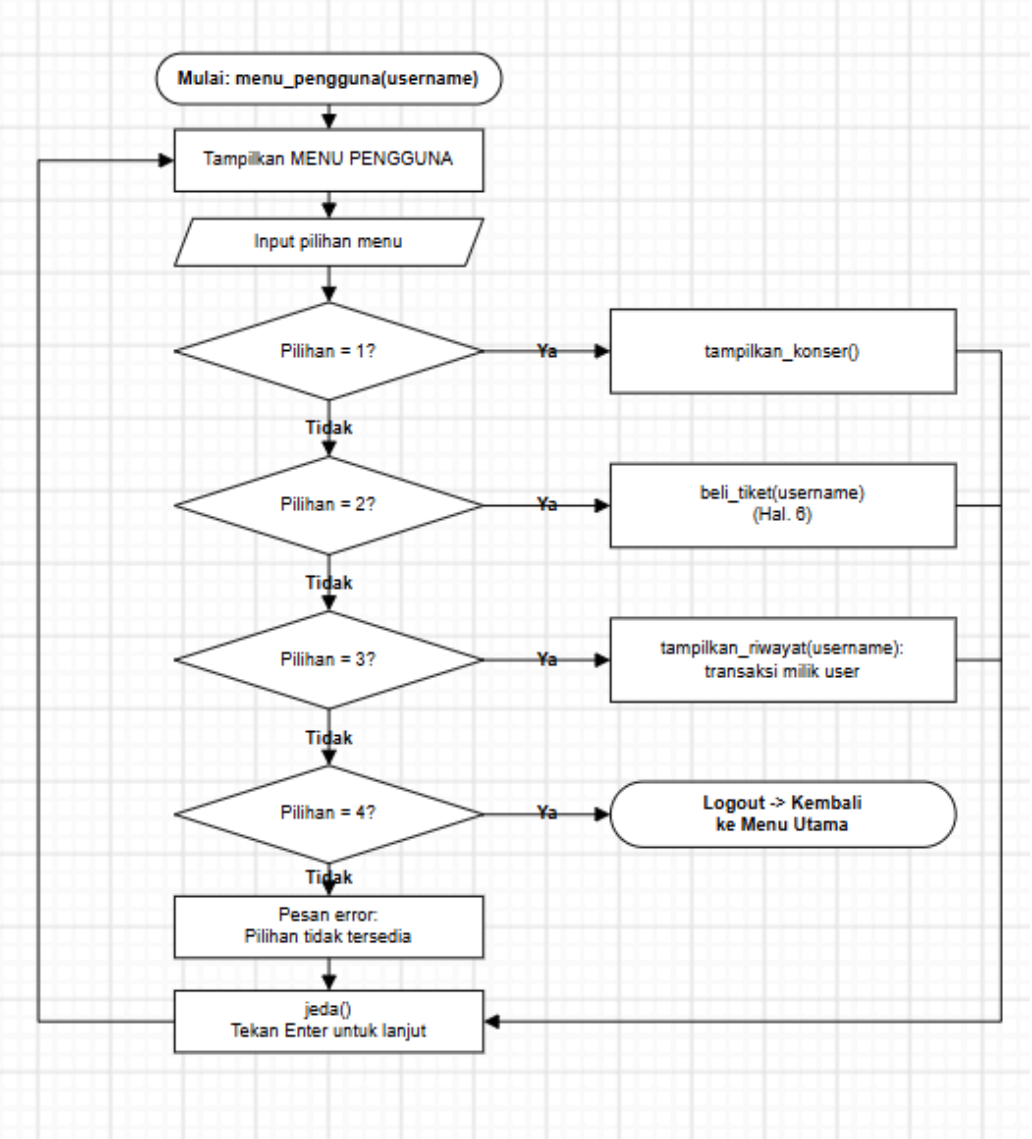
1. **Input:** Pengguna memasukkan username dan password untuk membuat akun baru.
2. **Proses:** Program memvalidasi input, yaitu memeriksa username dan password tidak kosong, serta username belum terdaftar. Jika valid, data akun baru disimpan ke dalam daftar pengguna.
3. **Output:** Menampilkan pesan “Registrasi berhasil” jika data valid, atau pesan kesalahan seperti “Username sudah terdaftar” atau “Input tidak boleh kosong” jika data tidak valid.



Gambar 1.4 Menu Admin

4. Menu Admin

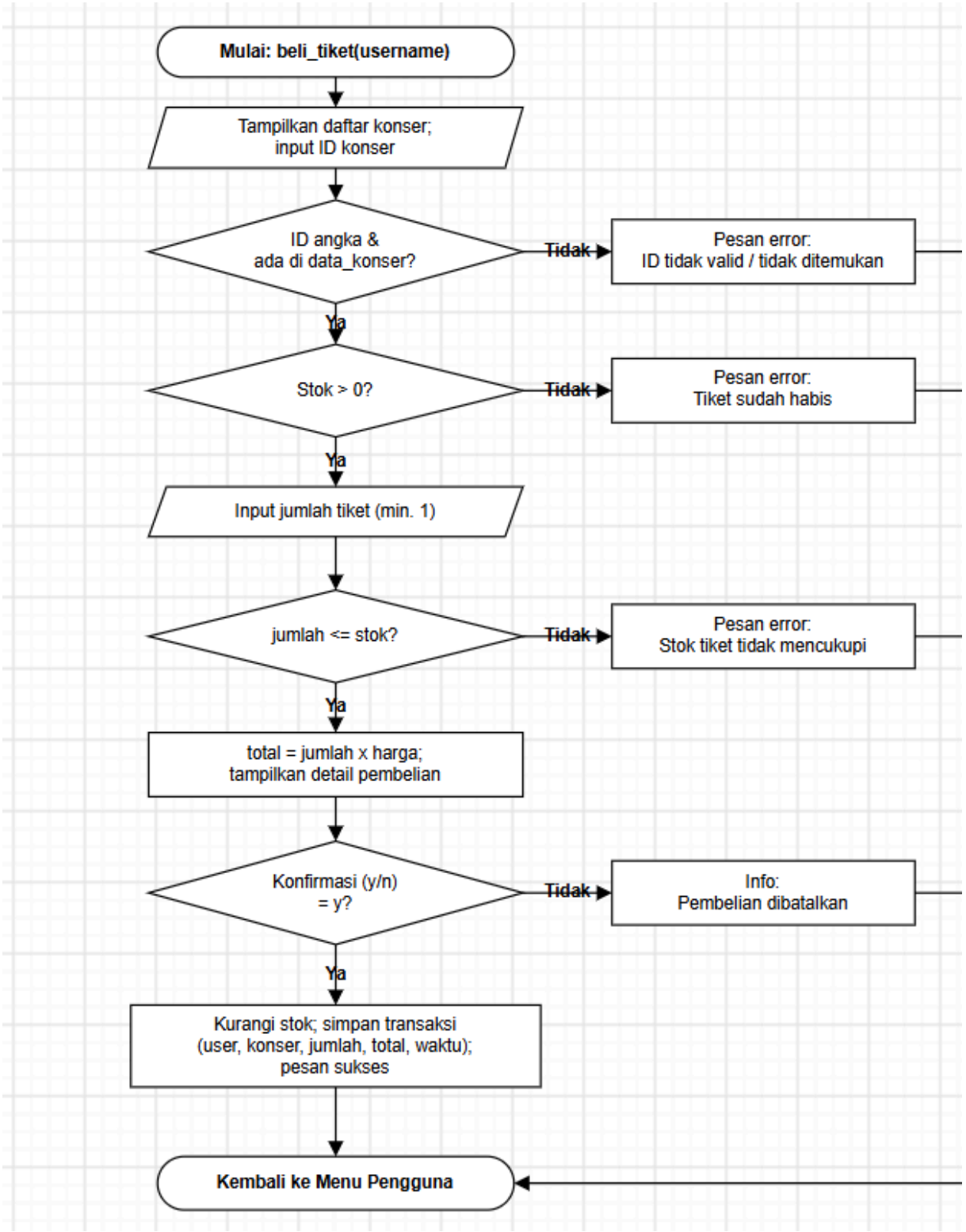
1. **Input:** Admin memasukkan pilihan menu_admin (1 sampai 5).
2. **Proses:** Program mengevaluasi pilihan menu admin untuk mengelola konser atau logout.
3. **Output:** Menampilkan daftar konser, membuka submenu (Tambah, Ubah, Hapus Konser), mencetak pesan logout, atau menampilkan pesan “Pilihan tidak tersedia”.



Gambar 1.5 Menu Pengguna

5. Menu Pengguna

1. **Input**: Pengguna memasukkan id_beli, jumlah tiket yang ingin dibeli, serta konfirmasi pembelian (y/n).
2. **Proses**: Program memvalidasi ketersediaan ID, memeriksa apakah stok masih ada dan mencukupi, menghitung total harga, serta meminta konfirmasi akhir.
3. **Output**: Jika dibatalkan atau stok tidak cukup, mencetak pesan gagal/pembatalan ke Menu Pengguna; jika berhasil, mengurangi stok konser, mencatat transaksi ke data_transaksi, mencetak "Pembelian berhasil", lalu kembali ke Menu Pengguna


Mulai: beli_tiket(username)
Tampilkan daftar konser;
input ID konser
ID angka &
ada di data_konser?
Tidak
Pesan error:
ID tidak valid / tidak ditemukan
Ya
Stok > 0?
Tidak
Pesan error:
Tiket sudah habis
Ya
Input jumlah tiket (min. 1)
jumlah <= stok?
Tidak
Pesan error:
Stok tiket tidak mencukupi
Ya
total = jumlah x harga;
tampilkan detail pembelian
Konfirmasi (y/n)
= y?
Tidak
Info:
Pembelian dibatalkan
Ya
Kurangi stok; simpan transaksi
(user, konser, jumlah, total, waktu);
pesan sukses
Kembali ke Menu Pengguna

Gambar 1.6 Beli Tiket

6. Beli Tiket
    1. **Input:** Pengguna memilih konser dari daftar yang tersedia, lalu memasukkan jumlah tiket yang ingin dibeli.
    2. **Proses:** Program memvalidasi pilihan konser dan jumlah tiket, yaitu memeriksa konser tersedia, jumlah berupa angka lebih dari 0, dan stok tiket mencukupi. Jika valid, program menghitung total harga (harga tiket × jumlah), mengurangi stok tiket, dan menyimpan transaksi ke riwayat pembelian pengguna.
    3. **Output:** Menampilkan pesan “Pembelian berhasil” beserta rincian pembelian (nama konser, jumlah tiket, dan total harga), atau pesan kesalahan seperti “Konser tidak ditemukan”, “Jumlah tiket tidak valid”, atau “Stok tiket tidak mencukupi”.

2. Deskripsi Singkat Program

Program ini merupakan simulasi aplikasi penjualan tiket konser digital berbasis teks yang dirancang untuk memfasilitasi dua peran pengguna utama, yaitu admin dan penonton umum. Administrator memiliki kontrol penuh untuk mengelola inventaris konser—mulai dari melihat daftar, menambah jadwal baru, memperbarui harga atau stok, hingga menghapus data konser yang ada. Dan juga, pengguna biasa dapat melakukan registrasi akun, menjelajahi konser yang tersedia, membeli tiket dengan sistem pengecekan stok otomatis yang langsung memotong jumlah ketersediaan, serta memantau riwayat transaksi pribadi mereka melalui menu interaktif yang berjalan di dalam konsol.

3. Source Code

```
from colorama import init
from tampilan import bersihkan_layar, jeda, tampilkan_menu, pesan_sukses,
pesan_error
from auth import proses_login, proses_registrasi
from admin import menu_admin
from pengguna import menu_pengguna


def main():
    while True:
        bersihkan_layar()
        tampilkan_menu("SISTEM PENJUALAN TIKET KONSER LANY",
                       ["Login", "Registrasi", "Keluar"])
        pilihan = input("Pilih menu: ")

        if pilihan == "1":
            username, peran = proses_login()
            if peran == "admin":
                jeda()
                menu_admin()
            elif peran == "user":
                jeda()
                menu_pengguna(username)
        elif pilihan == "2":
            proses_registrasi()
        elif pilihan == "3":
            pesan_sukses("Terima kasih telah menggunakan program.")
            break
        else:
            pesan_error("Pilihan tidak tersedia.")
        jeda()
```

```
from data import data_konser
from validasi import input_angka, input_teks
from perhitungan import buat_id_konser
from tampilan import (bersihkan_layar, jeda, tampilkan_konser, tampilkan_menu,
                      pesan_sukses, pesan_error)


def tambah_konser():
    nama = input_teks("Nama konser: ")
    kota = input_teks("Kota konser: ")
    harga = input_angka("Harga tiket: ")
    stok = input_angka("Jumlah tiket: ")
    id_baru = buat_id_konser()
    data_konser[id_baru] = {"nama": nama, "kota": kota, "harga": harga,
"stok": stok}
    pesan_sukses("Konser berhasil ditambahkan.")


def ubah_konser():
    tampilkan_konser()
    try:
        id_edit = int(input("Masukkan ID konser yang diubah: "))
        if id_edit not in data_konser:
            raise KeyError(id_edit)
    except ValueError:
        pesan_error("ID harus berupa angka.")
        return
    except KeyError:
        pesan_error("ID konser tidak ditemukan.")
        return
    nama = input_teks("Nama konser baru: ")
    kota = input_teks("Kota baru: ")
    harga = input_angka("Harga baru: ")
    stok = input_angka("Stok baru: ")
    data_konser[id_edit] = {"nama": nama, "kota": kota, "harga": harga,
"stok": stok}
    pesan_sukses("Data konser berhasil diubah.")


def hapus_konser():
    tampilkan_konser()
    try:
        id_hapus = int(input("Masukkan ID konser yang dihapus: "))
        del data_konser[id_hapus]
```

```
        pesan_sukses("Konser berhasil dihapus.")
    except ValueError:
        pesan_error("ID harus berupa angka.")
    except KeyError:
        pesan_error("ID konser tidak ditemukan.")


def menu_admin():
    while True:
        bersihkan_layar()
        tampilkan_menu("MENU ADMIN", [
            "Lihat Data Konser", "Tambah Konser", "Ubah Data Konser",
            "Hapus Konser", "Logout",
        ])
        pilihan = input("Pilih menu: ")
        if pilihan == "1":
            tampilkan_konser()
        elif pilihan == "2":
            tambah_konser()
        elif pilihan == "3":
            ubah_konser()
        elif pilihan == "4":
            hapus_konser()
        elif pilihan == "5":
            pesan_sukses("Logout berhasil.")
            break
        else:
            pesan_error("Pilihan tidak tersedia.")
        jeda()

    from datetime import datetime
from data import data_konser, data_transaksi
from validasi import input_angka
from perhitungan import buat_id_transaksi, hitung_total
from tampilan import (bersihkan_layar, jeda, tampilkan_konser,
tampilkan_riwayat,
                      tampilkan_detail_pembelian, tampilkan_menu,
                      pesan_sukses, pesan_error, pesan_info)


def beli_tiket(username):
    tampilkan_konser()
    try:
        id_beli = int(input("Masukkan ID konser: "))
        konser = data_konser[id_beli]
```

```
    except ValueError:
        pesan_error("ID harus berupa angka.")
        return
    except KeyError:
        pesan_error("ID konser tidak ditemukan.")
        return

    if konser["stok"] <= 0:
        pesan_error("Tiket sudah habis.")
        return

    jumlah = input_angka("Jumlah tiket: ", 1)
    if jumlah > konser["stok"]:
        pesan_error("Stok tiket tidak mencukupi.")
        return

    total = hitung_total(jumlah, konser["harga"])
    tampilkan_detail_pembelian(konser["nama"], jumlah, konser["harga"], total)
    konfirmasi = input("Konfirmasi pembelian? (y/n): ")
    if konfirmasi.lower() == "y":
        konser["stok"] -= jumlah
        data_transaksi[buat_id_transaksi()] = {
            "username": username,
            "konser": konser["nama"],
            "jumlah": jumlah,
            "total": total,
            "waktu": datetime.now().strftime("%d/%m/%Y %H:%M"),
        }
        pesan_sukses("Pembelian berhasil.")
    else:
        pesan_info("Pembelian dibatalkan.")


def menu_pengguna(username):
    while True:
        bersihkan_layar()
        tampilkan_menu("MENU PENGGUNA | " + username, [
            "Lihat Daftar Konser", "Beli Tiket", "Riwayat Transaksi",
"Logout",
        ])
        pilihan = input("Pilih menu: ")
        if pilihan == "1":
            tampilkan_konser()
        elif pilihan == "2":
            beli_tiket(username)
```

```
        elif pilihan == "3":
            tampilkan_riwayat(username)
        elif pilihan == "4":
            pesan_sukses("Logout berhasil.")
            break
        else:
            pesan_error("Pilihan tidak tersedia.")
        jeda()

data_konser = {
    1: {"nama": "LANY - Jakarta", "kota": "Jakarta", "harga": 850000, "stok":
100},
    2: {"nama": "LANY - Surabaya", "kota": "Surabaya", "harga": 750000,
"stok": 80},
    3: {"nama": "LANY - Bali", "kota": "Bali", "harga": 900000, "stok": 60},
}

data_user = {
    "admin": {"password": "admin123", "peran": "admin"},
}

data_transaksi = {}

# Penghitung ID terakhir (dictionary agar tidak perlu 'global')
counter = {"konser": 3, "transaksi": 0}

import os
from colorama import Fore
from prettytable import PrettyTable
from data import data_konser, data_transaksi


def bersihkan_layar():
    os.system("cls" if os.name == "nt" else "clear")


def jeda():
    input("\nTekan Enter untuk melanjutkan...")


def pesan_sukses(teks):
    print(Fore.GREEN + teks)


def pesan_error(teks):
```

```
    print(Fore.RED + teks)


def pesan_info(teks):
    print(Fore.YELLOW + teks)


def judul(teks):
    print(Fore.CYAN + "\n=== " + teks + " ===")


def tampilkan_konser():
    judul("DAFTAR KONSER")
    if len(data_konser) == 0:
        pesan_info("Belum ada data konser.")
        return
    tabel = PrettyTable()
    tabel.field_names = ["ID", "Nama", "Kota", "Harga (Rp)", "Stok"]
    for id_konser, konser in data_konser.items():
        tabel.add_row([id_konser, konser["nama"], konser["kota"],
                       konser["harga"], konser["stok"]])
    tabel.align["Nama"] = "l"
    tabel.align["Kota"] = "l"
    tabel.align["Harga (Rp)"] = "r"
    tabel.align["Stok"] = "r"
    print(Fore.BLUE + str(tabel))


def tampilkan_riwayat(username):
    judul("RIWAYAT TRANSAKSI")
    tabel = PrettyTable()
    tabel.field_names = ["No", "Waktu", "Konser", "Jumlah", "Total (Rp)"]
    no = 1
    for transaksi in data_transaksi.values():
        if transaksi["username"] == username:
            tabel.add_row([no, transaksi["waktu"], transaksi["konser"],
                           transaksi["jumlah"], transaksi["total"]])
            no += 1
    if no == 1:
        pesan_info("Belum ada transaksi.")
        return
    tabel.align["Konser"] = "l"
    tabel.align["Total (Rp)"] = "r"
    print(Fore.BLUE + str(tabel))
```

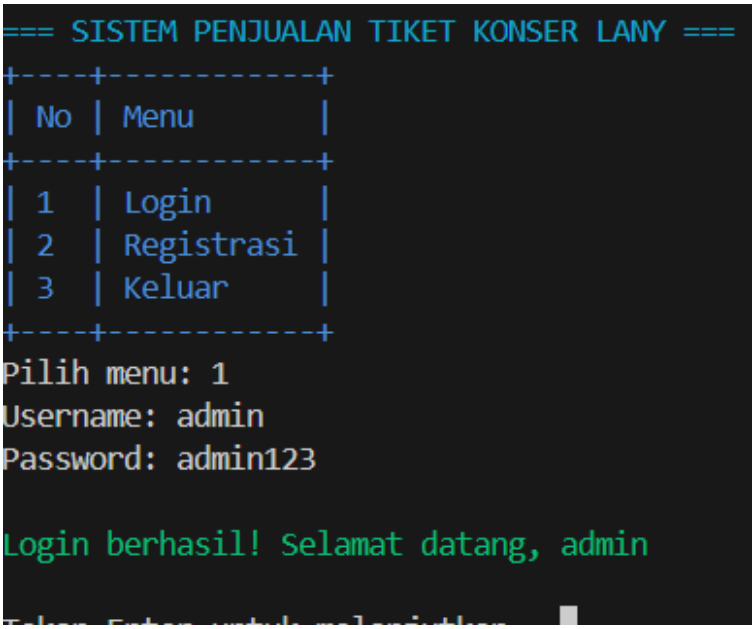
```python
def tampilkan_detail_pembelian(nama, jumlah, harga, total):
    tabel = PrettyTable()
    tabel.field_names = ["Konser", "Jumlah", "Harga (Rp)", "Total (Rp)"]
    tabel.add_row([nama, jumlah, harga, total])
    tabel.align["Konser"] = "l"
    print(Fore.BLUE + str(tabel))


def tampilkan_menu(judul_menu, daftar):
    """daftar: list string pilihan menu, ditampilkan sebagai tabel."""
    judul(judul_menu)
    tabel = PrettyTable()
    tabel.field_names = ["No", "Menu"]
    for i, teks in enumerate(daftar, start=1):
        tabel.add_row([i, teks])
    tabel.align["Menu"] = "l"
    print(Fore.BLUE + str(tabel))
```

4. Hasil Output

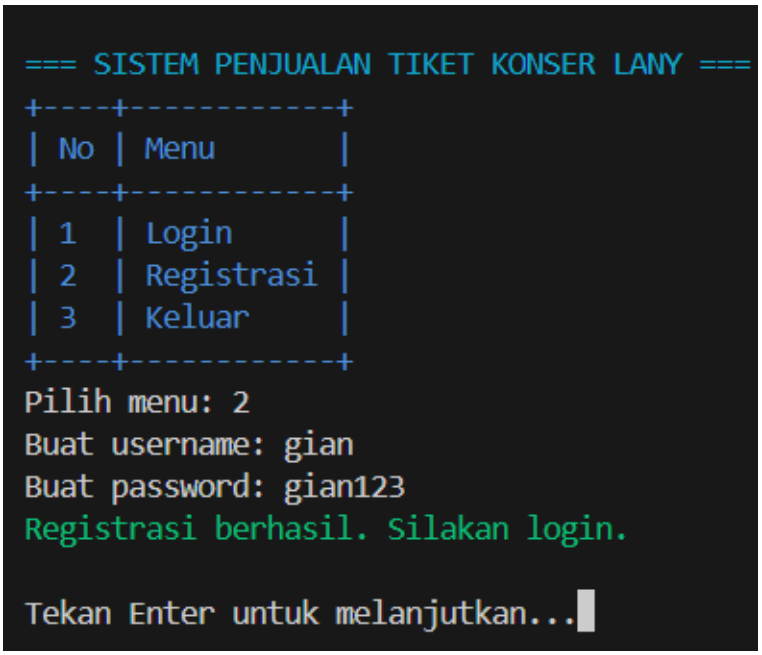
Berikut tangkapan layar hasil menjalankan program pada terminal.

Login menggunakan akun admin (username admin, password admin123) menampilkan Menu Admin.



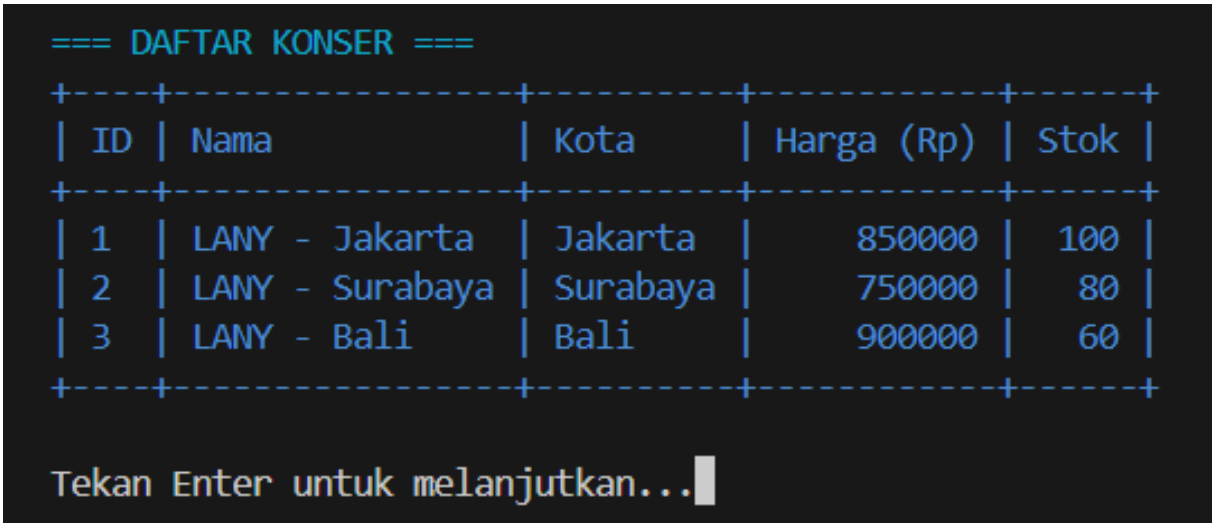
Gambar 4.1 Menu Admin Setelah Login

Pengguna baru mendaftar dengan username dan password, lalu program menampilkan pesan registrasi berhasil.



Gambar 4.2 Registrasi Akun

Admin memilih menu 1 untuk menampilkan seluruh data konser.



Gambar 4.3 Lihat Data Konser

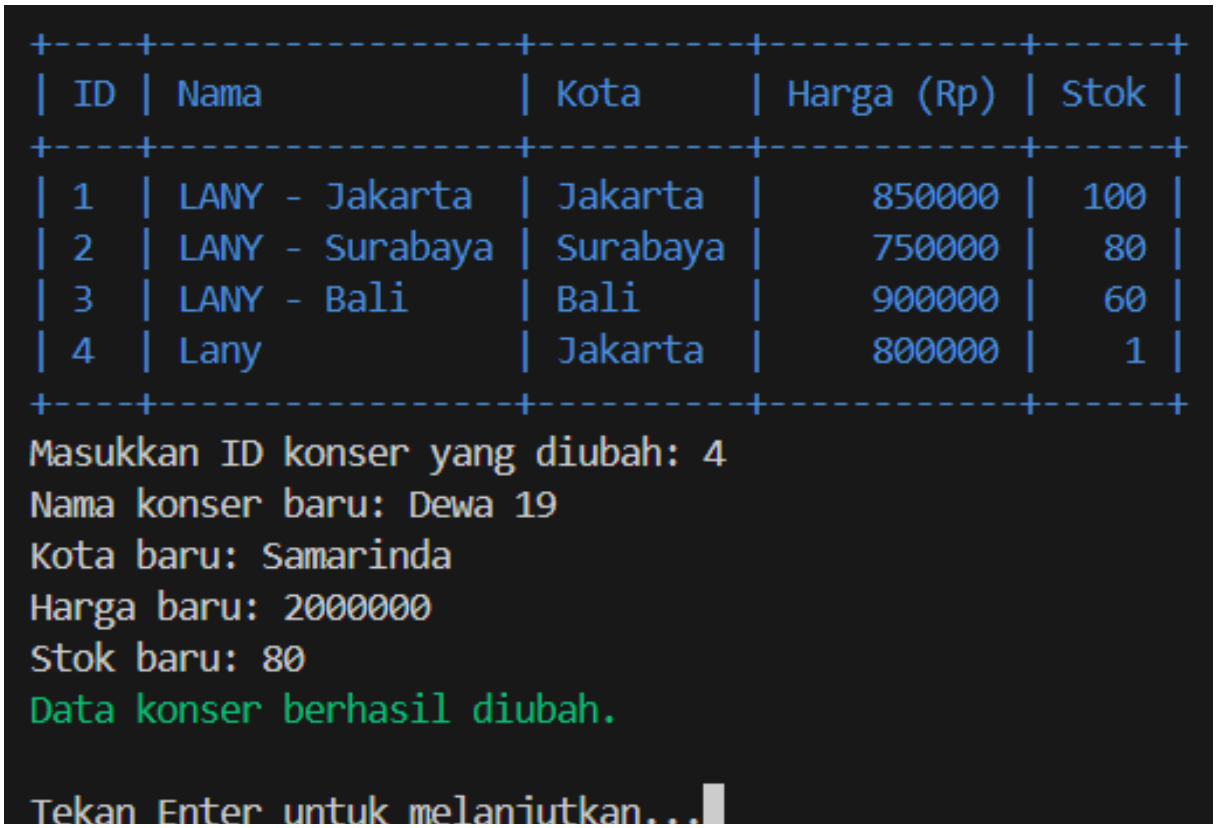
Admin menambahkan konser LANY - Bandung. ID baru dibuat otomatis (ID 4) dan data muncul pada daftar konser.

```
=== MENU ADMIN ===
+----+--------------------+
| No | Menu               |
+----+--------------------+
| 1  | Lihat Data Konser  |
| 2  | Tambah Konser      |
| 3  | Ubah Data Konser   |
| 4  | Hapus Konser       |
| 5  | Logout             |
+----+--------------------+
Pilih menu: 2
Nama konser: Lany
Kota konser: Jakarta
Harga tiket: 800000
Jumlah tiket: 1
Konser berhasil ditambahkan.

Tekan Enter untuk melanjutkan...
```

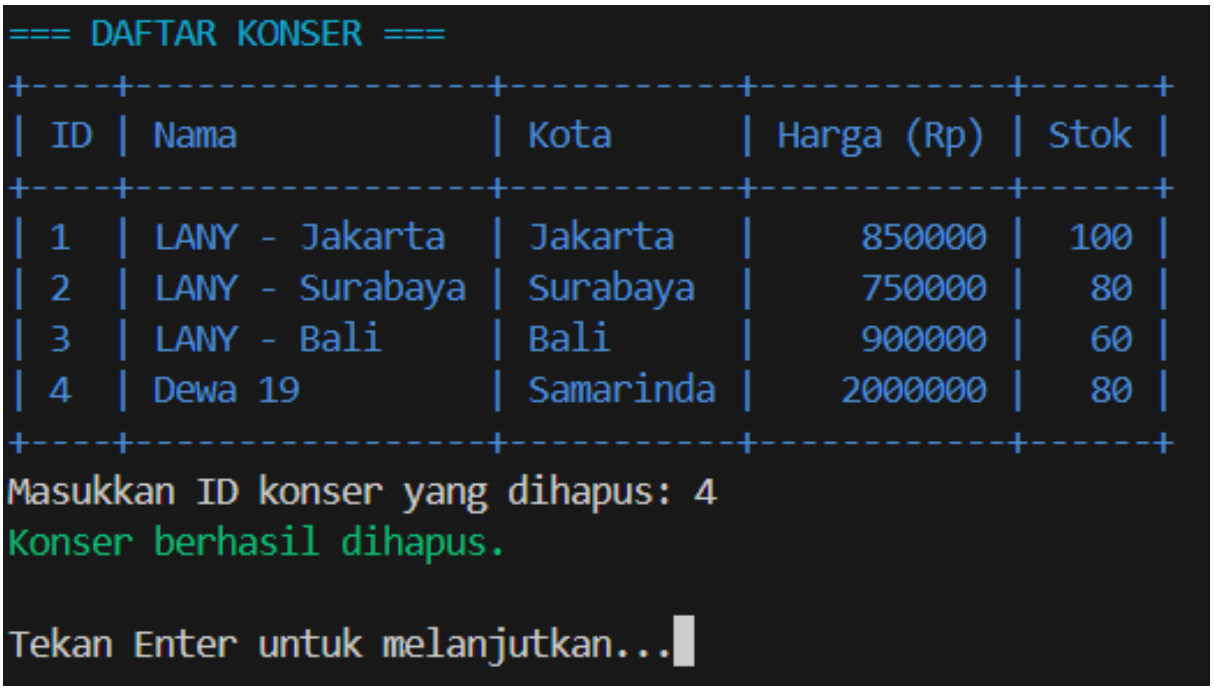
Gambar 4.4 Tambah Konser

Admin mengubah data konser dengan ID 2 (harga menjadi 2000000 dan stok menjadi 80). Hasil perubahan tampil pada daftar konser.



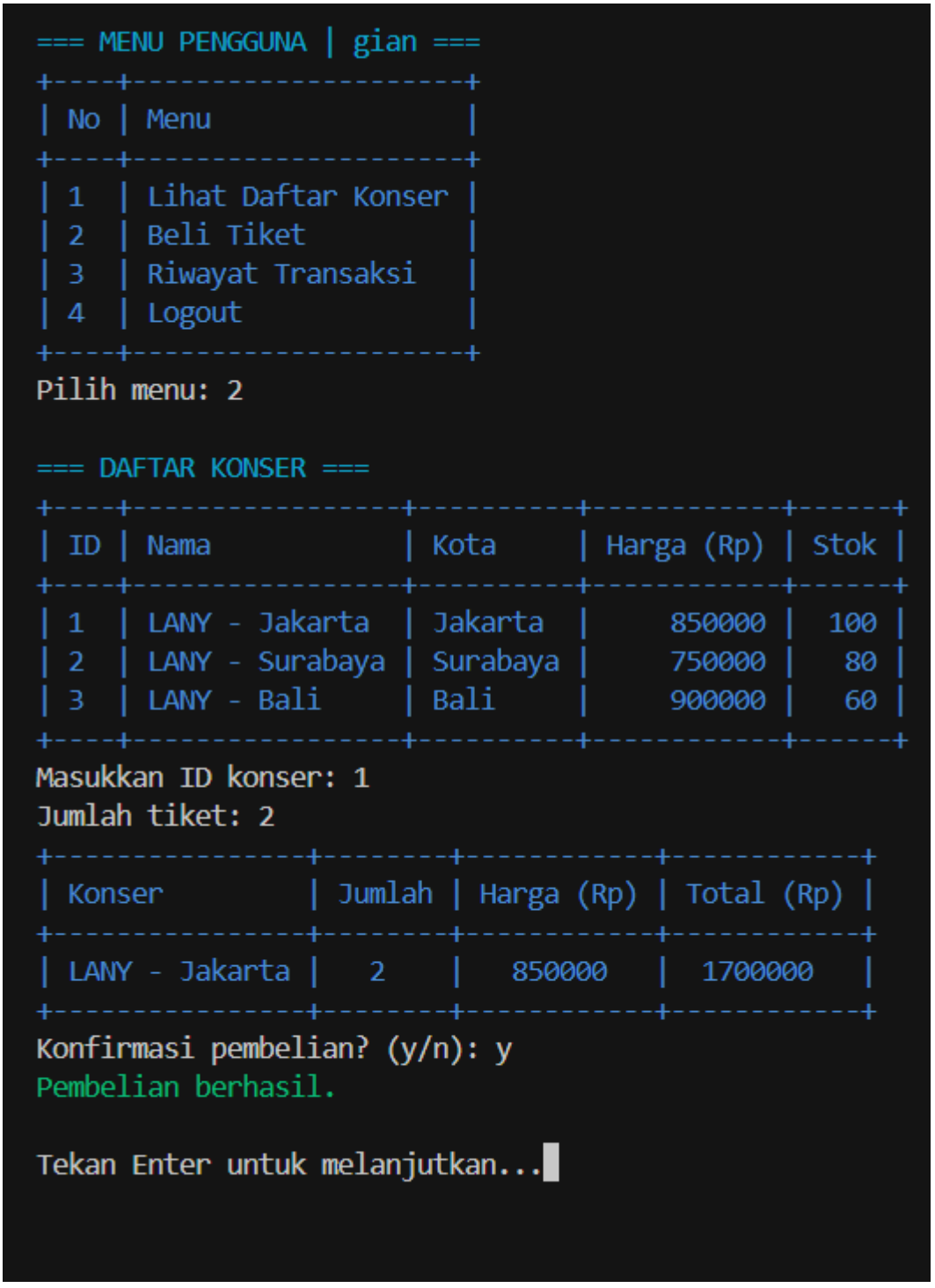
Gambar 4.5 Ubah Konser

Admin menghapus konser dengan ID 4. Setelah dihapus, konser tersebut tidak lagi muncul pada daftar.



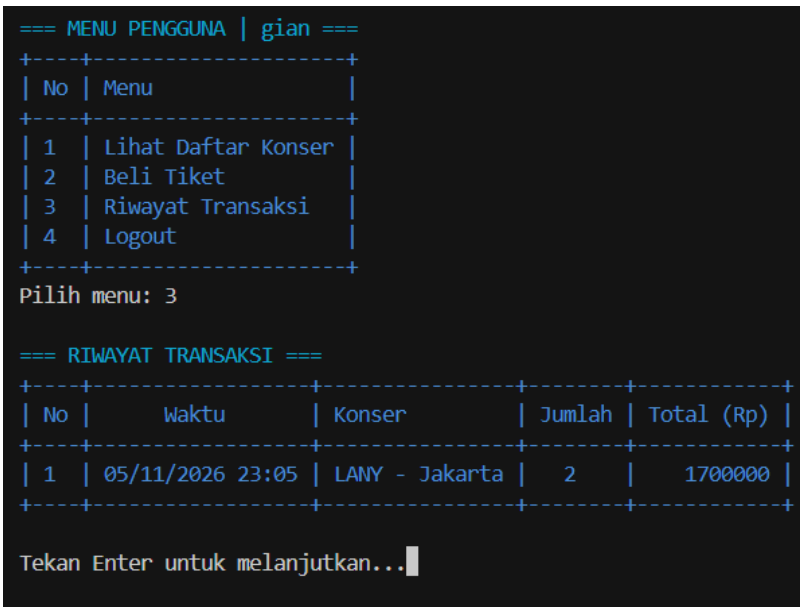
Gambar 4.6 Hapus Konser

Pengguna membeli 2 tiket konser LANY - Jakarta. Program menghitung total pembayaran dan meminta konfirmasi.



Gambar 4.7 Beli Tiket

Setelah pembelian, riwayat transaksi pengguna menampilkan konser, jumlah tiket, dan total pembayaran.



Gambar 4.8 Riwayat Transaksi

5. Langkah-langkah Git

5.1 Git Add

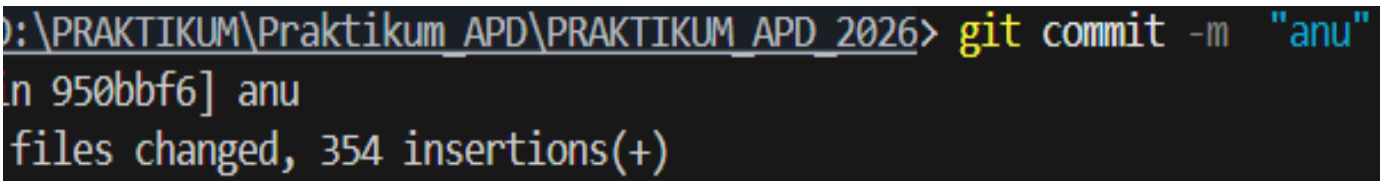
Perintah git add memasukkan file yang telah dibuat atau diubah (pada tugas ini file program Python) ke staging area, yaitu tahap persiapan sebelum perubahan disimpan dalam commit. Perintah git status digunakan untuk memastikan file sudah masuk staging.



Gambar 5.1 Git Add

5.2 Git Commit

Perintah git commit menyimpan semua perubahan yang ada di staging area ke dalam riwayat repository lokal sebagai satu versi. Opsi -m digunakan untuk menuliskan pesan yang menjelaskan isi perubahan.



Gambar 5.2 Git Commit

5.3 Git Push

Perintah git push mengirim commit dari repository lokal ke repository GitHub. origin adalah nama remote repository dan main adalah nama branch yang dituju. Setelah berhasil, file program dapat dilihat pada halaman repository GitHub.

```
PS D:\PRAKTIKUM\Praktikum_APD\PRAKTIKUM_APD_2026> git push
Enumerating objects: 38, done.
Counting objects: 100% (38/38), done.
Delta compression using up to 12 threads
Compressing objects: 100% (30/30), done.
Writing objects: 100% (35/35), 21.68 KiB | 2.71 MiB/s, done.
Total 35 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/gian14acdmx26-glitch/PRAKTIKUM_APD_2026.git
   b92c129..950bbf6  main -> main
PS D:\PRAKTIKUM\Praktikum_APD\PRAKTIKUM_APD_2026>
```

Gambar 5.3 Git Push